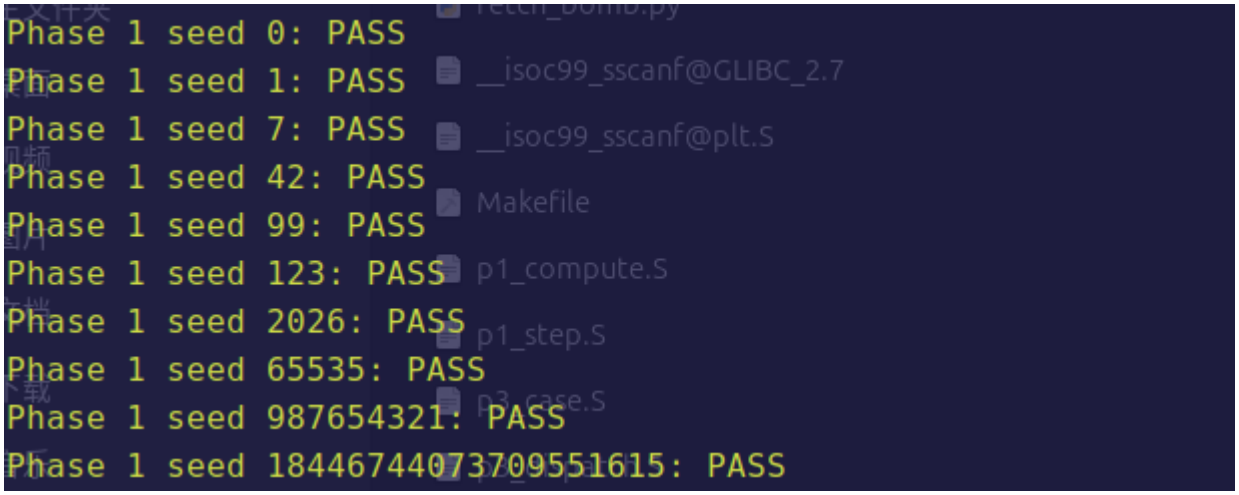
report　bomblab

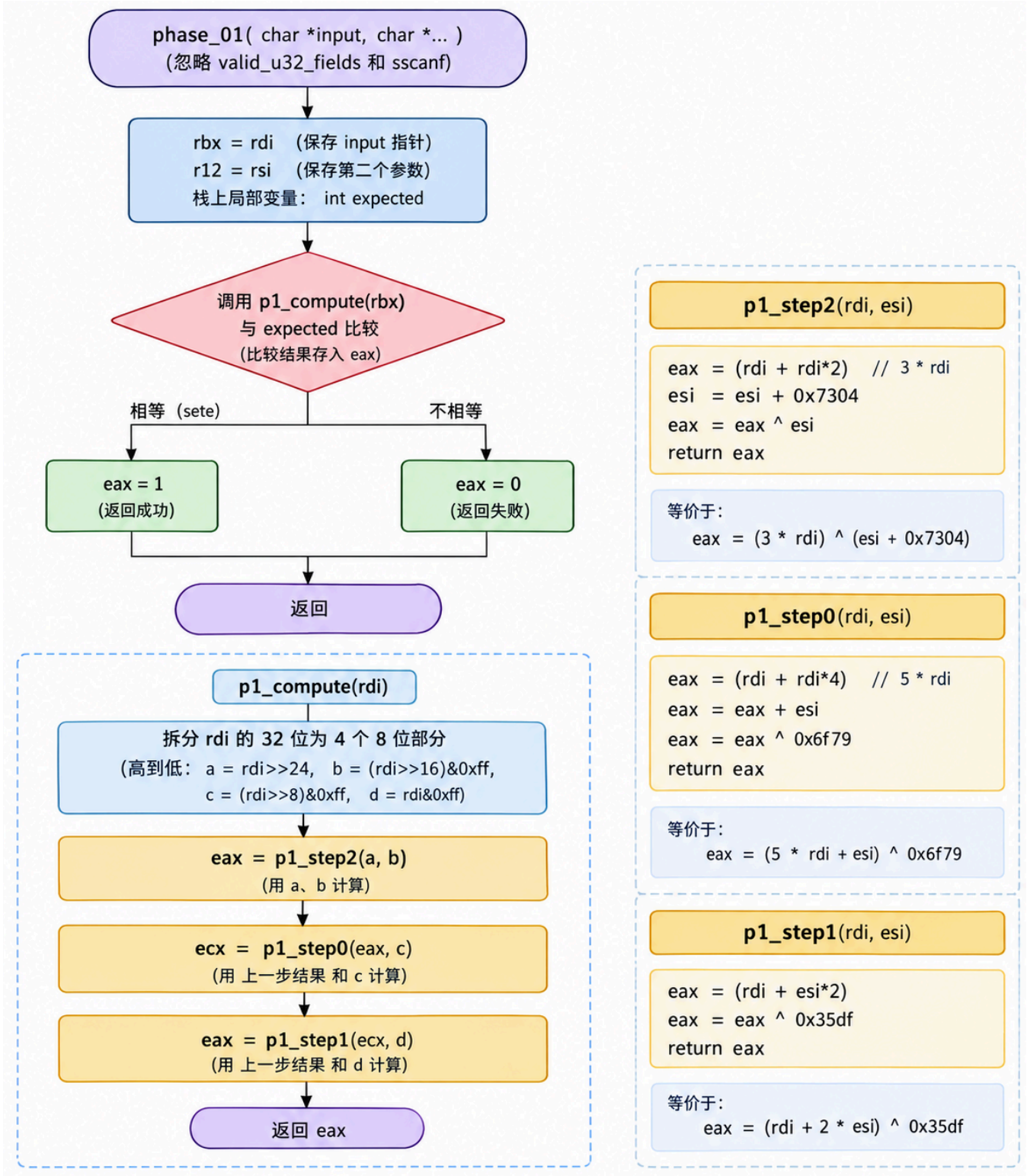
phase01

运行结果截图



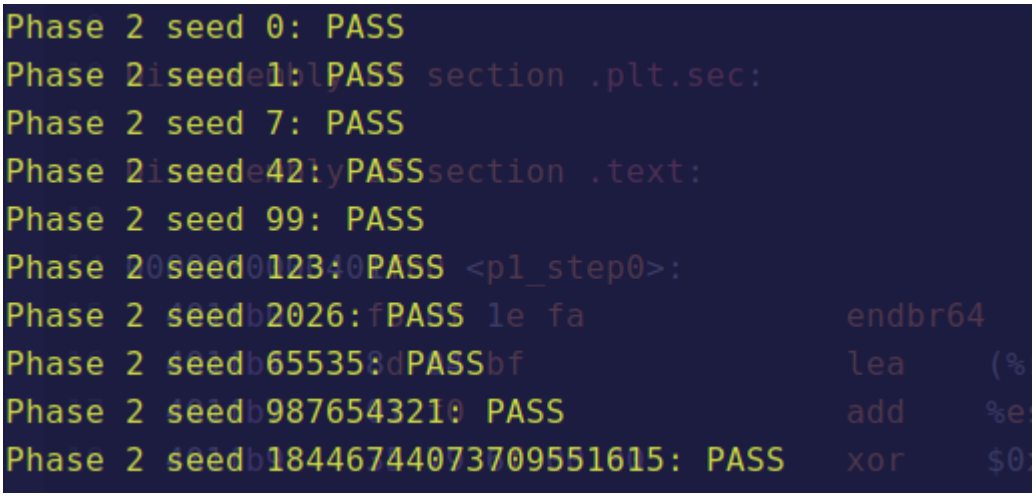
思路:

phase1首先建立栈帧并push需要使用的寄存器,调用valid_u32_fields和<_isoc99_sscanf@plt>对输入进行检测,通过调用__isoc99_sscanf@plt时传入的%rsi=1,得知外界输入一个参数,对于后面的phase和secret也有相似的操作,后面不再叙述.
phase1比较容易,根据phase_01中的call反汇编p1_compute和p1_steps,按流程走下去就可以得到答案,注意到challenge最初通过rsi传入


phase_01(char *input, char *...)
(忽略 valid_u32_fields 和 sscanf)
rbx = rdi (保存 input 指针)
r12 = rsi (保存第二个参数)
栈上局部变量: int expected
调用 p1_compute(rbx)
与 expected 比较
(比较结果存入 eax)
相等 (sete)
不相等
eax = 1
(返回成功)
eax = 0
(返回失败)
返回
p1_compute(rdi)
拆分 rdi 的 32 位为 4 个 8 位部分
(高到低: a = rdi>>24, b = (rdi>>16)&0xff,
c = (rdi>>8)&0xff, d = rdi&0xff)
eax = p1_step2(a, b)
(用 a、b 计算)
ecx = p1_step0(eax, c)
(用 上一步结果 和 c 计算)
eax = p1_step1(ecx, d)
(用 上一步结果 和 d 计算)
返回 eax
p1_step2(rdi, esi)
eax = (rdi + rdi*2) // 3 * rdi
esi = esi + 0x7304
eax = eax ^ esi
return eax
等价于:
eax = (3 * rdi) ^ (esi + 0x7304)
p1_step0(rdi, esi)
eax = (rdi + rdi*4) // 5 * rdi
eax = eax + esi
eax = eax ^ 0x6f79
return eax
等价于:
eax = (5 * rdi + esi) ^ 0x6f79
p1_step1(rdi, esi)
eax = (rdi + esi*2)
eax = eax ^ 0x35df
return eax
等价于:
eax = (rdi + 2 * esi) ^ 0x35df

phase02

运行结果截图



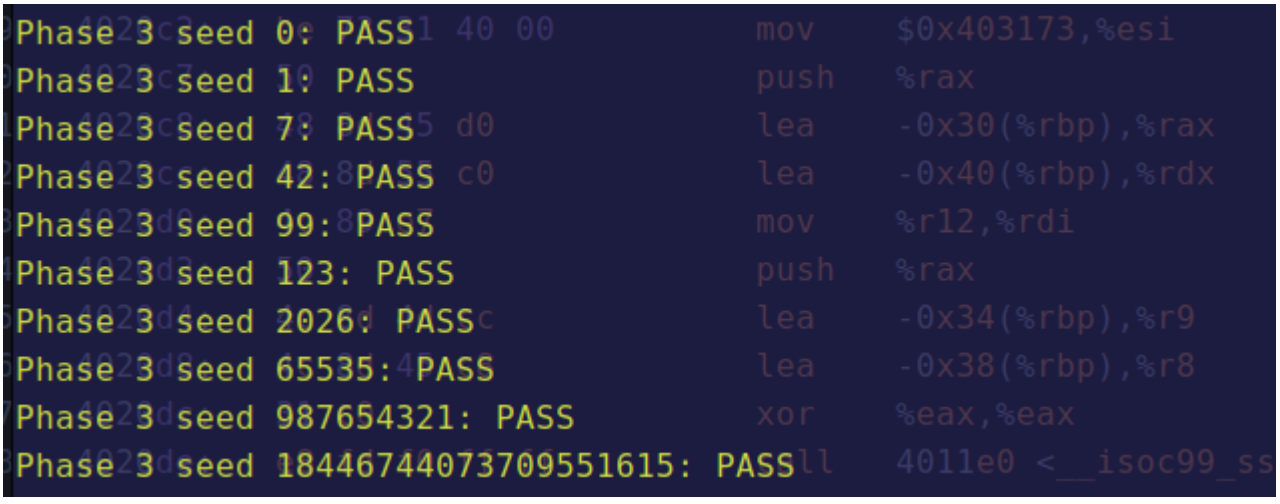
思路

输入6个参数,在调用sscanf前暴露了这些参数的存储地址是-(0x2c)%rbx到-(0x40)%rbx,从上到下是x6...x1,x1是可以直接确定的,x2...x6采用了循环的方法进行确定,和下面的代码等价

```
uint64_t x[6];
uint64_t i = 0xd, j = 0xf;
x[0] = (((challenge & 0xFFFFFFFF) & 0x7f) * 3) & 0xFFFFFFFF;
for (int p = 1; p <= 5; p++) {
  x[p] = ((((challenge >> j) & 0xf) ^ (2 * x[p - 1])) + i) & 0xFFFFFFFF;
  i += 0xd;
  j += 0x3;
}
```

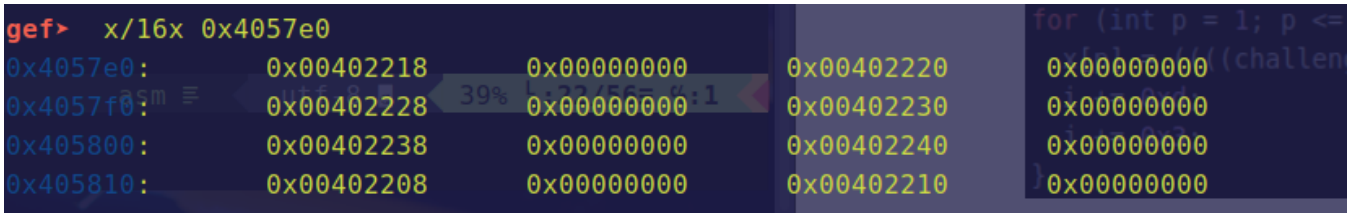
phase03

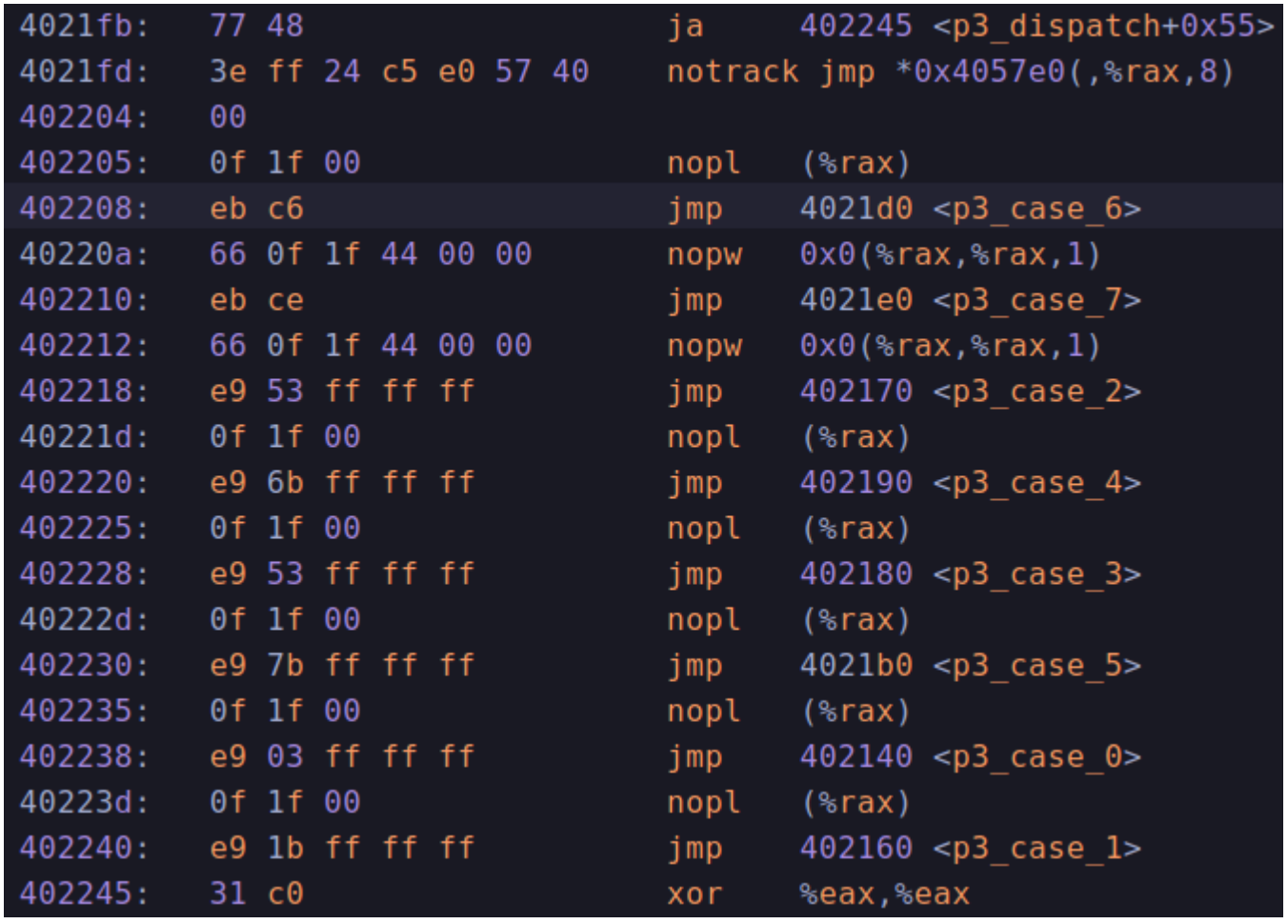
运行结果截图



思路

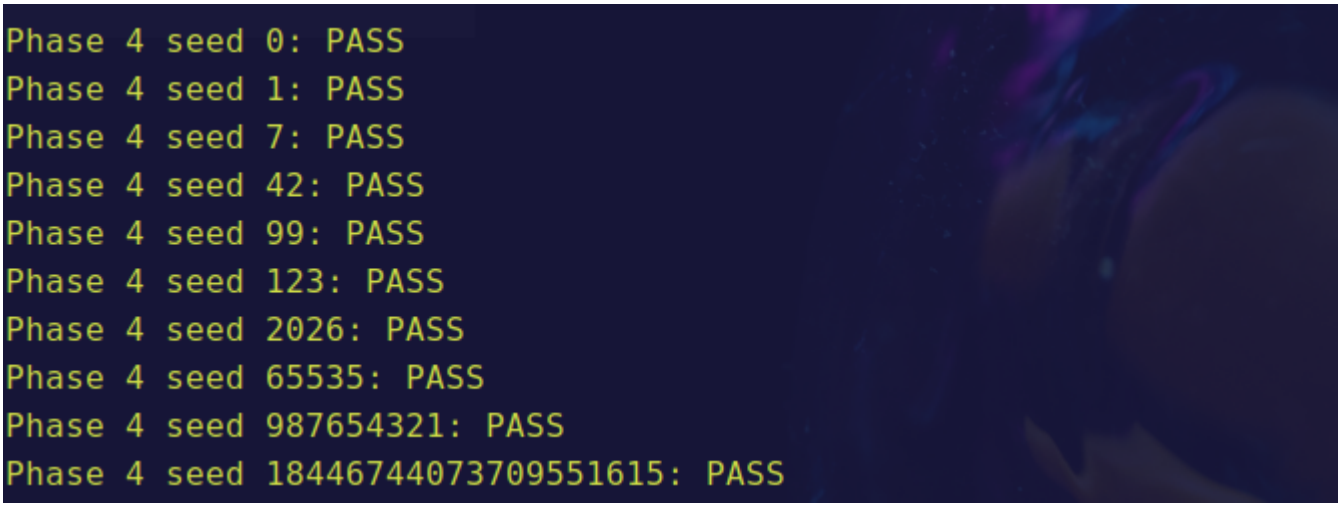
输入两个参数,地址分别是-0x14(%rbp),-0x18(%rbp),switch根据challenge&0x7的结果跳转到对应地址,地址的存储内容利用gdb查看,如下图,注意到实际的跳转判别和名称p3_case_i中的i并不相同,梳理跳转关系后可以得到第二个参数





phase04

运行结果如图



思路

输入1个参数.初步运算后进入p4_walk,p4_walk是一个递归函数,通过比较%rax和%rdi的值,分三条路径,相等时返回,其他情况各有自己的递归操作,复原后如下图,函数返回的结果就是要输入的参数

```
// 定义了一个辅助函数用于求解p4
uint64_t p4_walk_fake(uint64_t *rax, uint64_t *rdx, uint64_t *rdi,
                      uint64_t *rsi, uint64_t c) {
  *rax = (*rdi) & 0xFFFFFFFF;
  *rdi = ((*rdx - (*rsi & 0xFFFFFFFF)) >> 1) + (*rsi & 0xFFFFFFFF);
  if (*rax == *rdi) {
    *rax = (*rdi + 0x6) ^ c;
    return *rax;
  } else {
    uint64_t rbx;
    rbx = (*rdi + c) & 0xFFFFFFFF;
    if (*rax < *rdi) {
      *rdx = *rdi - 1;
      *rdi = *rax;
      p4_walk_fake(rax, rdx, rdi, rsi, c);
      *rax = ((5 * (*rax)) ^ rbx) & 0xFFFFFFFF;
      *rax = (*rax) ^ 0x1b;
      return *rax;
    } else {
      *rsi = *rdi + 1;
      *rdi = *rax;
      p4_walk_fake(rax, rdx, rdi, rsi, c);
      *rax = (((*rax) * 5) ^ rbx) & 0xFFFFFFFF;
      *rax = (*rax) ^ 0x19;
      return *rax;
    }
  }
}
```

phase05

运行结果如图

```
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
```

思路

输入两个参数.p5考察struct,第一步访问的是struct指针数组p5_heads,首地址0x405b20,再根据该数组元素的值,访问对应地址并将对应的值存入%rdx,通过查看p5_heads,进一步找到p5_nodes.观察发现nodes才是实际的struct,包括一个地址名和两个整数,nodes存储的地址指向另一个nodes,进一步观察发现heads并不包括所有的nodes,于是需要在C代码中重建对应关系.

p5_traverse采用了循环,一并考虑

```
1: i/p5_traverse.S
   4023c7:    48 89 f9                mov    %rdi,%rcx
   4023ca:    49 89 f0                mov    %rsi,%r8
   4023cd:    83 e0 07                and    $0x7,%eax
   4023d0:    48 c1 e9 03             shr    $0x3,%rcx
   4023d4:    48 8b 14 c5 20 5b 40    mov    0x405b20(,%rax,8),%rdx
```

```
0x405b20 <p5_heads>:    0x00405bb0      0x00000000      0x00405ba0      0x00000000
0x405b30 <p5_heads+16>: 0x00405be0      0x00000000      0x00405b70      0x00000000
0x405b40 <p5_heads+32>: 0x00405bc0      0x00000000      0x00405b80      0x00000000
0x405b50 <p5_heads+48>: 0x00405c00      0x00000000      0x00405c10      0x00000000
```

```
0x405b60 <p5_nodes>:    0x00000000      0x00000000      0x00000850      0x00008627
0x405b70 <p5_nodes+16>: 0x00405bc0      0x00000000      0x00000161      0x00007ca5
0x405b80 <p5_nodes+32>: 0x00405c00      0x00000000      0x0000065c      0x0000dc34
0x405b90 <p5_nodes+48>: 0x00405bd0      0x00000000      0x0000037a      0x00003ef3
gef➤
0x405ba0 <p5_nodes+64>: 0x00405be0      0x00000000      0x00000b78      0x000014f1
0x405bb0 <p5_nodes+80>: 0x00405ba0      0x00000000      0x00000444      0x000019a3
0x405bc0 <p5_nodes+96>: 0x00405b80      0x00000000      0x00000986      0x000039ed
0x405bd0 <p5_nodes+112>:        0x00405b60      0x00000000      0x0000001e      0x000037d7
gef➤
0x405be0 <p5_nodes+128>:        0x00405b70      0x00000000      0x00000547      0x0000126b
0x405bf0 <p5_nodes+144>:        0x00405b90      0x00000000      0x0000076b      0x00007257
0x405c00 <p5_nodes+160>:        0x00405c10      0x00000000      0x0000023f      0x000044e3
0x405c10 <p5_nodes+176>:        0x00405bf0      0x00000000      0x00000a2e      0x0000d80a
```

```
struct node d[12] = {
    [0]={.link=NULL, .n1=0x850, .n2=0x8627},    // d[0]
    [1]={.link=d + 6, .n1=0x161, .n2=0x7ca5},   // d[1]
    [2]={.link=d + 10, .n1=0x65c, .n2=0xdc34}, // d[2]
    [3]={.link=d + 7, .n1=0x37a, .n2=0x3ef3},   // d[3]
    [4]={.link=d + 8, .n1=0xb78, .n2=0x14f1},   // d[4]
    [5]={.link=d + 4, .n1=0x444, .n2=0x19a3},   // d[5]
    [6]={.link=d + 2, .n1=0x986, .n2=0x39ed},   // d[6]
    [7]={.link=d, .n1=0x1e, .n2=0x37d7},        // d[7]
    [8]={.link=d + 1, .n1=0x547, .n2=0x126b},   // d[8]
    [9]={.link=d + 3, .n1=0x76b, .n2=0x7257},   // d[9]
    [10]={.link=d + 11, .n1=0x23f, .n2=0x44e3}, // d[10]
    [11]={.link=d + 9, .n1=0xa2e, .n2=0xd80a}   // d[11]
};
struct node *p5_heads[8] = {[0]=d + 5, [1]=d + 4, [2]=d + 8,  [3]=d + 1,
                            [4]=d + 6, [5]=d + 2, [6]=d + 10, [7]=d + 11};
```

phase06

运行结果

```
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
```

思路

输入一个参数.进入p6_dispatch后,发现函数指针的使用,检查相应的地址,导出p6_ops.S,作对应(发现地址 并不是等差数列),然后按照C的写法构造函数和相应的函数指针数组,就可以完成

```
gef➤  x/16x 0x405dc0
0x405dc0 <p6_ops>:      0x00402500      0x00000000      0x004024f0      0x00000000
0x405dd0 <p6_ops+16>:   0x004024a0      0x00000000      0x004024b0      0x00000000
0x405de0 <p6_ops+32>:   0x004024d0      0x00000000      0x004024e0      0x00000000
0x405df0 <p6_ops+48>:   0x004024a0      0x00000000      0x00402500      0x00000000
```

```c
void p6_op0(uint64_t *a, uint64_t *d, uint64_t *dx) {
  (void)dx;
  *a = *d;
  *a = (*a >> 2) | (*a << 30);
  *a = (*a) & 0xFFFFFFFF;
}

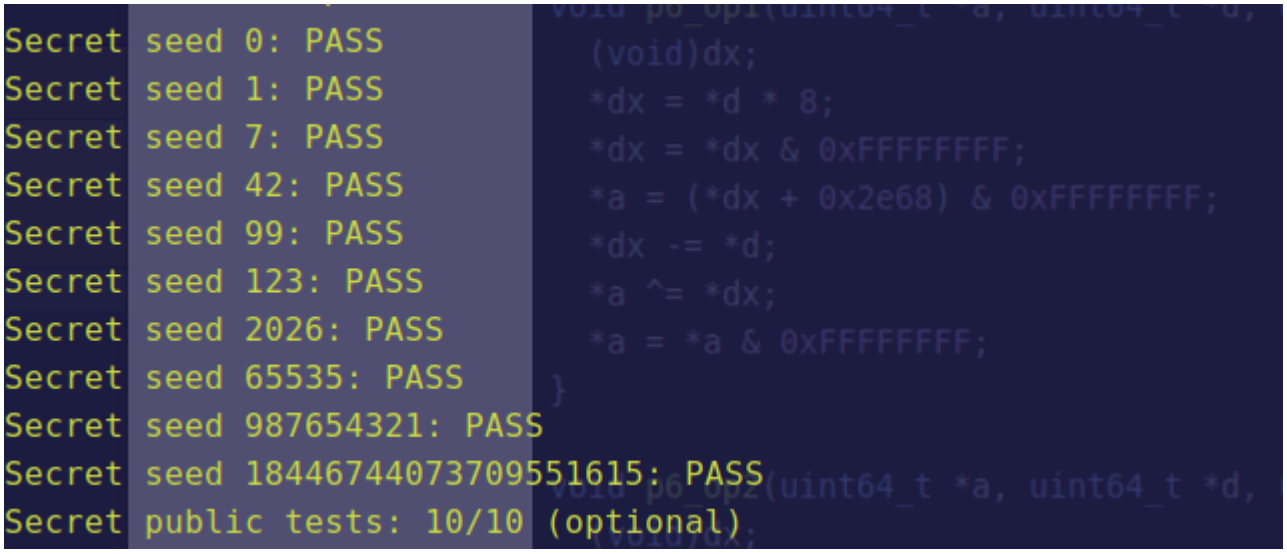
void p6_op1(uint64_t *a, uint64_t *d, uint64_t *dx) {
  (void)dx;
  *dx = *d * 8;
  *dx = *dx & 0xFFFFFFFF;
  *a = (*dx + 0x2e68) & 0xFFFFFFFF;
  *dx -= *d;
  *a ^= *dx;
  *a = *a & 0xFFFFFFFF;
}

void p6_op2(uint64_t *a, uint64_t *d, uint64_t *dx) {
  (void)dx;
  *a = *d;
  *a = (*a >> 1) ^ (*d & 0xFFFFFFFF);
  *a += 0x973e;
  *a = (*a) & 0xFFFFFFFF;
}

void p6_op3(uint64_t *a, uint64_t *d, uint64_t *dx) {
  (void)dx;
  *a = *d ^ 0x7062;
  *a = (*a + 8 * (*d)) & 0xFFFFFFFF;
}
void p6_op4(uint64_t *a, uint64_t *d, uint64_t *dx) {
  (void)dx;
  *a = *d;
  *a = ((*a << 5) | (*a >> 27)) & 0xFFFFFFFF;
  *a ^= 0xbfac;
}
void p6_op5(uint64_t *a, uint64_t *d, uint64_t *dx) {
  (void)dx;
  *a = ((*d) + 0x506c) & 0xFFFFFFFF;
  *d = (*d) >> 3;
  *a = (*d) ^ (*a);
```

secret

运行结果



思路

输入以32空格开头,跟随`OPEN x1 x2 x3`,然后一共有`gate`,`copy`,`phase`,`mix`四个主要部分.注意到有`challenge`和`context`两个参数需要使用.`gate`会检查输入长度,要求`36<=length<=163`,并且会将内存上的相应位置全部清零.随后调用`copy`,将`sourse`写入内存.随后`gate`会检查写入后的字符串第33字节起是否为`OPEN`,最后使用`phase`验证后面的三个参数,对于后面的参数,按序分析即可

```
47   402716:   e8 75 ff ff ff          call   402690 <secret_copy>
48   40271b:   81 bd 50 ff ff ff 4f    cmpl   $0x4e45504f,-0xb0(%rbp)
```

其他seed

```
(base) waldein@waldein-OMEN-Gaming-Laptop-16-am0xxx:~/ics-2026-bomblab$ ./defuser --seed 67 | ./bomb --seed 67
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]
Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
```

AI使用情况

1. 在p3梳理switch跳转地址,p5梳理struct指针数组的地址对应关系,p6梳理函数指针数组和函数的对应关系时,采用AI辅助处理数据,解放双眼劳动
2. 在secret中,由于笨人不会使用snprintf,借助AI编写了将结果写入out的代码